



APRA

Adaptive Progressive Robust Aggregation for Backdoor-Resilient Federated Learning








APRA 

1623201

162320127 162320115 162320129



<https://github.com/GreenInsect/APRA>



1.	2019	2
2.	2019	3
2.1.	2019	3
2.2.	2019	3
3.	2019	3
4.	2019	4
5.	2019	4
5.1.	2019	4
5.2.	APRA 2019	5
5.3.	2019	5
5.4.	2019	6
5.5.	2019	6
5.6.	2019	6
6.	2019	7
7.	2019	7
7.1.	2019	7
7.2.	2019	9
7.3.	APRA 2019	11
8.	2019	12
8.1.	2019	12
8.2.	2019	12
8.3.	2019	13
8.4.	2019	13
8.5.	2019	14
8.6.	NBD/NDIF 2019	14
8.7.	PCA 2019	15
8.8.	2019 MAD 2019	16
8.9.	2019	16
8.10.	2019	16
8.11.	2019	17
8.12.	2019	17
8.13.	2019	18
8.14.	2019	19
9.	2019	20
10.	2019	21



APRA *Adaptive Progressive Robust Aggregation* CIFAR-10 A3FL DOBA ReBA Neurotoxin APRA 700 PPT

```

Requirements.md fl_utils/apra.py fl_utils/aggregator.py
main/yamls/cifar10_apra.yaml main/re_result main/re_result_pre APRA_Presentation_AI.md
APRA_Presentation_meaningful_images ASR 700 APRA
main/re_result apra_round_summary.csv
```

FedAvg

$$w_{t+1} = w_t + \frac{1}{n} \sum_{i=1}^n \Delta w_i$$

FedAvg Clip FoolsGold Sybil Non-IID DeepSight RFLBAT

APRA

实验内容：Non-IID 数据集上的 APRA 实验

实验环境：APRA 实验环境

实验步骤

APRA 实验环境

- 1. L2 正则化 NBD/NDIF PCA
- 2. MAD
- 3.
- 4.

APRA 实验环境

实验结果

实验结果

实验结果	实验结果
fl_utils/apra.py	APRA 实验环境
fl_utils/aggregator.py	FedAvgAPRAClipDeepSightFoolsGoldRFLBAT
fl_utils/fler.py	
fl_utils/attacker.py	A3FLReBANeurotoxin
main/yamls/cifar10_apra.yaml	CIFAR-10 实验环境 APRA
main/re_result*	ASR APRA

fl_utils/aggregator.py 实验环境 aggregator APRAAggregator.aggregate

```

if self.helper.config['agg_method'] == 'avg':
    return self.average_models(...)
elif self.helper.config['agg_method'] == 'apra':
    apraAggregator = APRAAggregator(self.helper)
    return apraAggregator.aggregate(...)
elif self.helper.config['agg_method'] == 'clip':
    self.clip_updates(...)
    return self.average_shrink_models(...)

```

附录A APRA 附录A

APRA 附录A

APRA 附录A

附录A w_t 附录A $\{\Delta w_i\}$ 附录A $\{m_i\}$ 附录A S
 附录A w_{t+1}

1. 附录A i 附录A f_i 附录A n_i
2. 附录A $\text{median}(n)$ 附录A $\text{MAD}(n)$ 附录A modified z-score
3. 附录A $k_t = \max(2, k_0 \exp(-\lambda t))$ 附录A
4. 附录A
5. 附录A “附录A” 附录A
6. 附录A
7. 附录A
8. 附录A MAD 附录A

附录A MAD 附录A

$$z_i = 0.6745 \frac{n_i - \text{median}(n)}{\text{MAD}(n)}$$

$$k_t = \max(2.0, k_{\text{init}} \exp(-k_{\text{decay}} t))$$

附录A k_{init} 附录A k_{decay} 附录A

附录A

附录A

_extract_features 附录A L2 附录A CIFAR-10 附录A get_layer_name_for_dataset 附录A
 附录A NBD/NDIF 附录A NBD 附录A NDIF 附录A
 附录A

```

flat = flatten_update(update, layer_names=[layer_name])
flat_updates.append(flat)

```

```

l2 = 0.0
for name, data in update.items():
    if 'num_batches_tracked' in name:
        continue
    l2 += torch.norm(data, p=2).item() ** 2
update_norms.append(np.sqrt(l2))

```

 “” “”  PCA 



MAD  APRA 


```

for k in range(2, max_k + 1):
    clustering = AgglomerativeClustering(
        n_clusters=k, metric='euclidean', linkage='ward'
    )
    labels = clustering.fit_predict(features)
    score = silhouette_score(features, labels)

for c in np.unique(labels):
    indices = np.where(labels == c)[0]
    internal_sim = np.mean(sk_cosine_similarity(features[indices]))
    cluster_scores[c] = len(indices) * internal_sim

```

 10 



 APRA 


```

ratio = (w_i + 1e-8) / (max_trust + 1e-8)
clip_factors[i] = base_clip * ratio

update = client_weight[name].float() * trust_weights[idx_in_list]
averaged_weights[name] += update

```

 APRA  MAD 



APRA CSV apra_client_trace.csv MAD
apra_round_summary.csv

CIFAR-10 ResNet18 700 100 10
5 Non-IID Dirichlet 0.7

- 1. Main Task Accuracy
- 2. Attack Success Rate ASR
- 3. APRA

FedAvg Clip DeepSight FoolsGold RFLBAT APRA A3FL DOBA ReBA Neurotoxin
700

	APRA	FedAvg	Clip	DeepSight	FoolsGold	RFLBAT
A3FL	90.3862	92.2713	92.2840	91.8031	92.3456	92.2495
DOBA	90.7134	92.1527	92.1648	91.7948	92.3430	92.2688
Neurotoxin	89.8676	88.2799	91.1706	87.3673	87.1985	89.5529
ReBA	91.7909	92.4428	92.4738	91.9591	91.8411	92.4053
	90.6895	91.2867	92.0233	90.7311	90.9321	91.6191

APRA_Presentation_AI.md

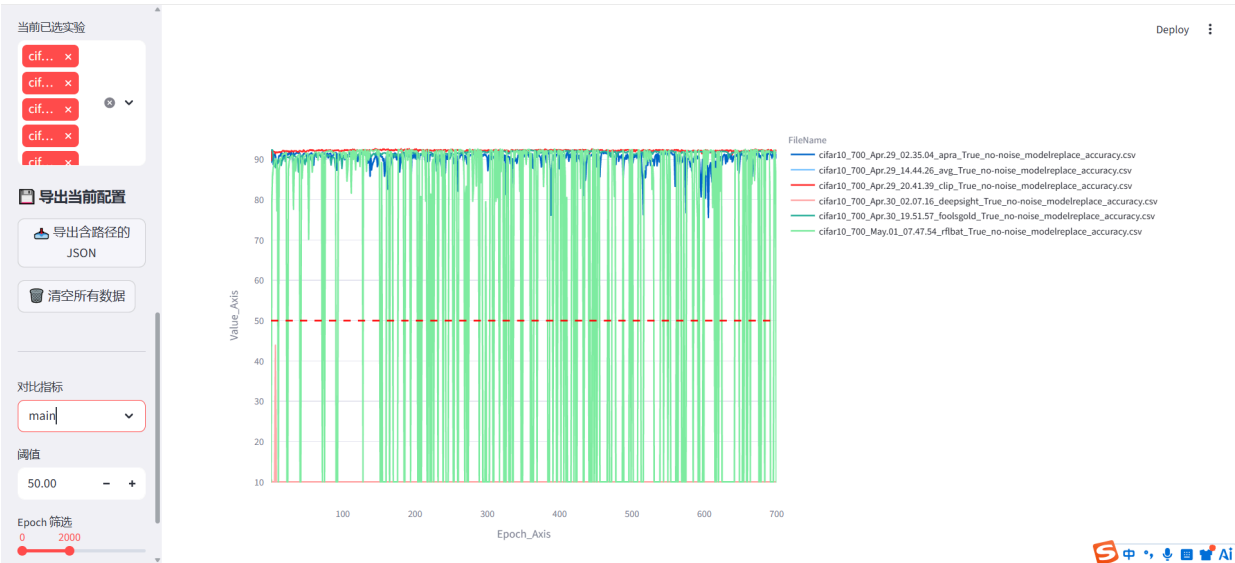
APRA 90.6895% Clip RFLBAT 90% APRA
Neurotoxin APRA 89.8676% FedAvg DeepSight FoolsGold



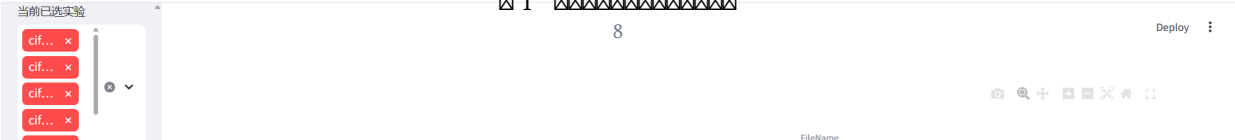
A3FL



DOBA



ReBA

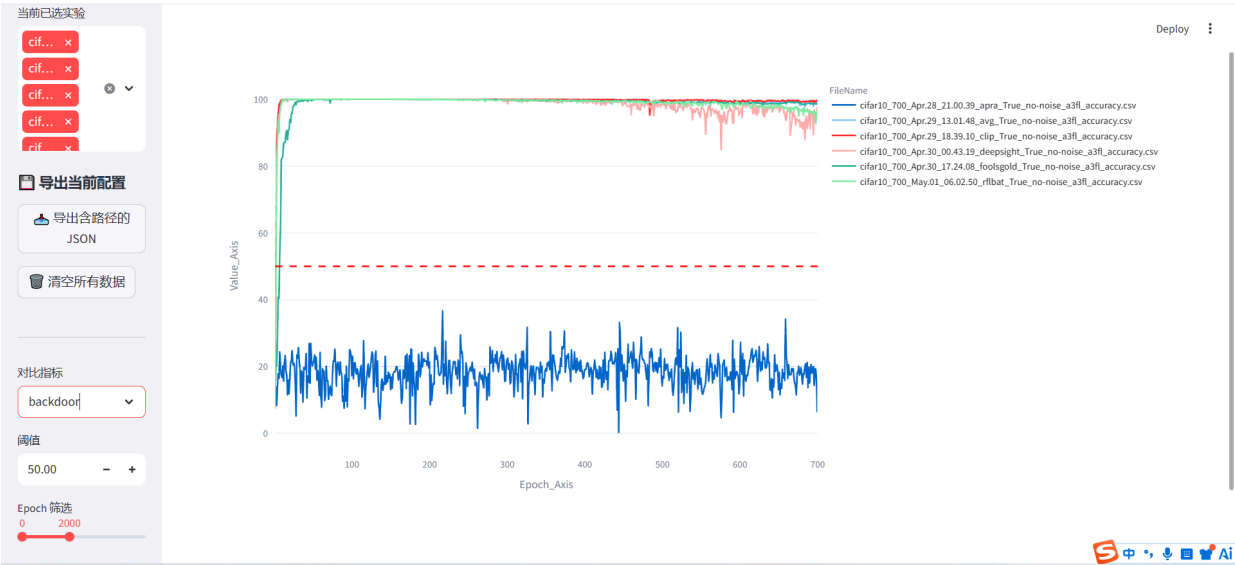


APRA ASR

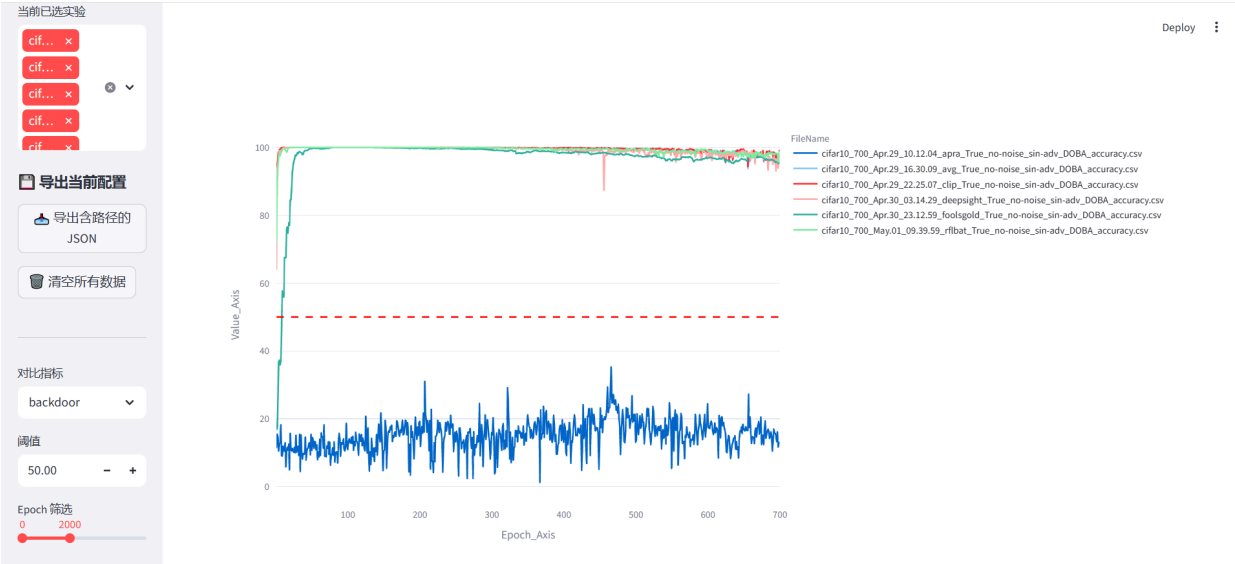
	APRA	FedAvg	Clip	DeepSight	FoolsGold	RFLBAT
A3FL	18.4313	99.4797	99.5790	98.3359	98.5094	98.9970
DOBA	15.2119	99.5345	99.5334	99.1195	97.1416	99.3758
Neurotoxin	9.4279	97.4656	94.1472	91.1622	99.5117	98.0828
ReBA	13.5486	60.6899	60.7015	53.8362	54.6519	20.1430
	14.1549	89.2924	88.4903	85.6135	87.4537	79.1497

APRA_Presentation_AI.md ASR

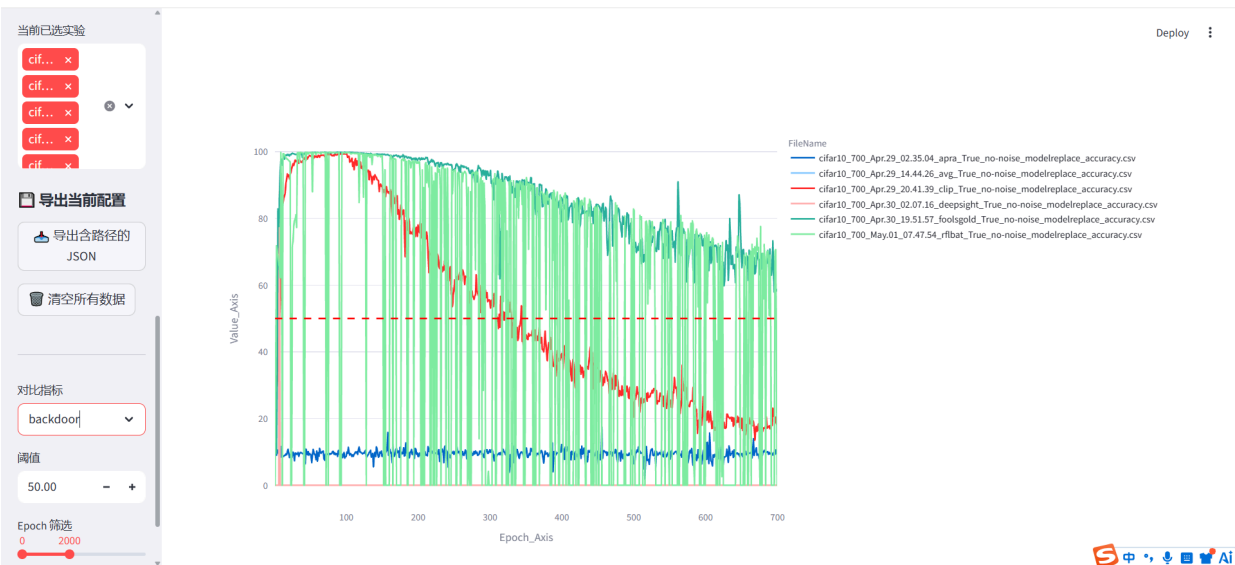
ASR APRA ASR 14.1549% FedAvg 89.2924% Neurotoxin
APRA ASR 9.4279% ReBA RFLBAT ASR 20.1430%
APRA 13.5486%



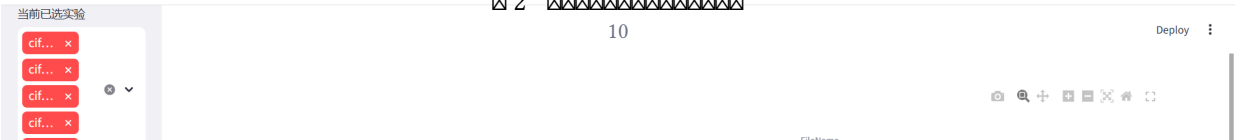
A3FL



DOBA



ReBA



```

XXXXXXXXXX main/re_result XXXX APRA XXXXXXXXXXXXXXX 700 XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

```

Model	Size	Time	Acc	ASR	Time	Acc	ASR
A3FL	700	476/981	48.52%	51.48%	31.17%	90.42	19.34
ModelReplace	700	981/981	100.00%	0.00%	28.59%	90.48	9.48
DOBA	700	451/981	45.97%	54.03%	33.54%	90.51	14.94
Neurotoxin	700	504/989	50.96%	49.04%	33.80%	89.83	9.38

ModelReplace APRA 700 A3FL DOBA Neurotoxin ASR **APRA** “”7“”

[REDACTED]

[REDACTED]

[REDACTED]

fl_utils/aggregator.py [REDACTED] agg [REDACTED] agg_method [REDACTED]

```

if self.helper.config['agg_method'] == 'avg':
    return self.average_models(...)
elif self.helper.config['agg_method'] == 'apra':
    apraAggregator = APRAAggregator(self.helper)
    return apraAggregator.aggregate(
        global_model, weight_accumulator,
        weight_accumulator_by_client,
        client_models, sampled_participants, epoch
    )
elif self.helper.config['agg_method'] == 'clip':
    self.clip_updates(...)
    return self.average_shrink_models(...)

```

[REDACTED]

```

[REDACTED] APRA [REDACTED] FedAvg [REDACTED] Clip [REDACTED] DeepSight [REDACTED] FoolsGold [REDACTED] RFLBAT [REDACTED] APRA [REDACTED]
[REDACTED]

```

[REDACTED]

```

fl_utils/fler.py [REDACTED] weight_accumulator_by_client [REDACTED] APRA [REDACTED]
[REDACTED]

```

```

for participant_id in sampled_participants:
    model = self.helper.local_model
    self.copy_params(model, global_model_copy)

    if not self.if_adversary(epoch, participant_id) and adv_index == -1:
        self.train_benign(participant_id, model, epoch)
    else:
        if self.helper.config["attacker_method"] == 'neurotoxin':
            self.neurotoxin_train(participant_id, model, epoch, mask_grad_list)

```


[illegible]APRA NBD/NDIF PCA NaN Inf

10 PCA

MAD

MAD z-score

```
k = max(2.0, k_init * np.exp(-k_decay * epoch))
median_norm = np.median(update_norms)
mad = np.median(np.abs(update_norms - median_norm))

modified_z_scores = 0.6745 * (update_norms - median_norm) / mad
mask = np.abs(modified_z_scores) <= k
```

MAD k epoch

MAD APRA

```
if mask.sum() < max(2, n_clients // 2):
    sorted_indices = np.argsort(np.abs(modified_z_scores))
    keep_count = max(2, n_clients // 2)
    mask[:] = False
    mask[sorted_indices[:keep_count]] = True
    safety_keep_used = True
```

“” Non-IID

MAD APRA

```
for k in range(2, max_k + 1):
    clustering = AgglomerativeClustering(
        n_clusters=k, metric='euclidean', linkage='ward'
    )
    labels = clustering.fit_predict(features)
    score = silhouette_score(features, labels)
    if score > best_score:
        best_score = score
        best_k = k
```



```
cs = sk_cosine_similarity(features)
np.fill_diagonal(cs, -1.0)
maxcs = np.max(cs, axis=1) + epsilon

wv = 1 - np.max(cs, axis=1)
wv = np.clip(wv, 0, 1)
wv = wv / np.max(wv)
wv[wv == 1] = 0.99

wv = np.log(wv / (1 - wv) + epsilon) + 0.5
wv = np.clip(wv, 0, 1)
wv = 1.0 - wv
wv = wv / np.sum(wv)
```



```
for i in range(n):
    w_i = trust_weights[i]
    ratio = (w_i + 1e-8) / (max_trust + 1e-8)
    clip_factors[i] = base_clip * ratio

for key in update:
```



```

})

summary = {
    "epoch": int(epoch),
    "sampled_malicious_ids": self._json_list(...),
    "mad_reject_malicious_ids": self._json_list(...),
    "cluster_selected_cluster": int(selected_cluster),
    "final_selected_ids": self._json_list(final_selected_ids),
    "final_rejected_ids": self._json_list(final_rejected_ids),
}

```

配置项“`apra_base_clip`”用于配置基线模型 MAD 模型的 clip 系数，
配置项“`apra_k_init`”用于配置基线模型 MAD 模型的 clip 系数。

APRA 配置

APRA 配置项 `apra_pca_components`、`apra_base_clip`、`apra_k_init`、`apra_k_decay` 位于 `fl_utils/apra.py`，`apra_use_nbd_ndif` 位于 `main/yamls/cifar10_apra.yaml`，`apra_use_neup_ddif` 位于 `main/yamls/cifar10_apra.yaml`。

```

if config['apra_use_nbd_ndif'] and client_models is not None:
    nbd_ndif = self._extract_nbd_ndif(...)

```

配置

配置项 `APRA` 用于配置 NBD/NDIF 模型，配置项 `APRA` 位于 `main/yamls/cifar10_apra.yaml`。



APRA achieves the highest accuracy on CIFAR-10, achieving 90.69% accuracy, while ASR is 14.15%. FedAvg, Clip, DeepSight, FoolsGold, RFLBAT, APRA, Neurotoxin, and ReBA are compared.

APRA achieves the highest accuracy on CIFAR-10, achieving 90.69% accuracy, while ASR is 14.15%. FedAvg, Clip, DeepSight, FoolsGold, RFLBAT, APRA, Neurotoxin, and ReBA are compared.

APRA achieves the highest accuracy on CIFAR-10, achieving 90.69% accuracy, while ASR is 14.15%. FedAvg, Clip, DeepSight, FoolsGold, RFLBAT, APRA, Neurotoxin, and ReBA are compared.

APRA achieves the highest accuracy on CIFAR-10, achieving 90.69% accuracy, while ASR is 14.15%. FedAvg, Clip, DeepSight, FoolsGold, RFLBAT, APRA, Neurotoxin, and ReBA are compared.



-
1. McMahan, B., Moore, E., Ramage, D., Hampson, S., & Arcas, B. A. Communication-Efficient Learning of Deep Networks from Decentralized Data. AISTATS, 2017. <https://arxiv.org/abs/1602.05629>
 2. Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D., & Shmatikov, V. How To Backdoor Federated Learning. AISTATS, 2020. <https://proceedings.mlr.press/v108/bagdasaryan20a.html>
 3. Fung, C., Yoon, C. J. M., & Beschastnikh, I. The Limitations of Federated Learning in Sybil Settings. RAID, 2020. <https://arxiv.org/abs/1808.04866>
 4. Rieger, P., Nguyen, T. D., Miettinen, M., & Sadeghi, A. R. DeepSight: Mitigating Backdoor Attacks in Federated Learning Through Deep Model Inspection. NDSS, 2022. <https://arxiv.org/abs/2201.00763>
 5. Wang, Y., Zhai, D., Zhan, Y., & Xia, Y. RFLBAT: A Robust Federated Learning Algorithm against Backdoor Attack. arXiv, 2022. <https://arxiv.org/abs/2201.03772>
 6. Zhang, Z., Panda, A., Song, L., Yang, Y., Mahoney, M. W., Gonzalez, J. E., Ramchandran, K., & Mittal, P. Neurotoxin: Durable Backdoors in Federated Learning. ICML, 2022. <https://proceedings.mlr.press/v162/zhang22w.html>
 7. Zhang, H. et al. A3FL: Adversarially Adaptive Backdoor Attacks to Federated Learning. NeurIPS, 2023. <https://openreview.net/forum?id=S6ajVZy6FA>